

Instituto Politécnico Superior “Gral. San Martín”
Tecnicatura en Informática Profesional y Personal
Mantenimiento de Software
Docente: Paula Ávila

Ingeniería del Mantenimiento de Software

Estudiantes: Santiago Cañal
Nicolas Paz Reyes
Valentino Aviani
Martin Moloeznik

Fecha: mayo 2026

1 Introducción

El mantenimiento de software constituye una de las disciplinas fundamentales dentro de la ingeniería de software moderna. Lejos de ser una actividad secundaria o meramente correctiva, representa el conjunto de procesos sistemáticos que garantizan la supervivencia, la adaptabilidad y el valor estratégico de los sistemas informáticos a lo largo de su ciclo de vida operativo.

El presente informe de investigación aborda las definiciones fundamentales del mantenimiento de software, las clasificaciones estandarizadas de sus actividades, las métricas económicas que evidencian su impacto organizacional, los roles involucrados en su ejecución y los conceptos técnicos asociados a esta disciplina.

2 El Concepto de Mantenimiento de Software: Definición y Síntesis Metodológica

Una vez consolidada la ingeniería de software a raíz de la crisis de complejidad de fines de los años 60, se hizo evidente que la entrega inicial de un programa era apenas el comienzo de un ciclo continuo de modificaciones destinadas a asegurar su supervivencia y utilidad.

Para resolver las directrices académicas propuestas, se contrastan formalmente dos de las definiciones institucionales más respetadas y vigentes en el campo de la ingeniería de software:

- **Norma ANSI/IEEE 610.12 (1990):** Define el mantenimiento como la modificación de un producto de software después de su entrega al cliente o usuario para corregir defectos, para mejorar el rendimiento u otras propiedades deseables, o para adaptarlo a un cambio de entorno.
- **Norma ISO/IEC 14764 (2006):** Describe el mantenimiento como un proceso que describe el esqueleto de actividades destinadas a modificar un producto de software existente preservando su integridad funcional y arquitectónica. Este proceso cubre todo el ciclo de vida del sistema, desde la planificación de la transición del desarrollo inicial hasta el retiro definitivo del producto.

A partir de estas referencias, se sintetiza una definición propia y unificada:

El mantenimiento de software es el proceso de ingeniería que, posterior a la entrega del sistema, modifica su código y arquitectura para corregir defectos, adaptarlo a entornos cambiantes y mejorar su mantenibilidad y funcionalidad, preservando su estabilidad operativa.

3 Clasificación de las Actividades de Mantenimiento

Para poder planificar y controlar adecuadamente el esfuerzo de los ingenieros, es imprescindible agrupar las tareas de mantenimiento de acuerdo con criterios estandarizados. La literatura científica y los estándares internacionales dividen estas actividades en función de la causa raíz o la intencionalidad del cambio introducido en el sistema.

3.1 Tipología de Lientz y Swanson

Propuesta originalmente en la década de 1970, esta taxonomía se centra en las motivaciones del negocio y el usuario final:

- **Correctiva:** Modificaciones reactivas obligadas por fallos de ejecución detectados post-entrega, donde el programa no satisface los requerimientos iniciales.
- **Adaptativa:** Cambios exigidos para mantener la operatividad del sistema cuando se producen alteraciones en su entorno tecnológico externo (como actualizaciones del sistema operativo o cambios en bases de datos), sin modificar la funcionalidad percibida.
- **Perfectiva:** Tareas proactivas orientadas a expandir la funcionalidad solicitada por el negocio, optimizar el rendimiento de ejecución, depurar la documentación técnica o mejorar la experiencia del usuario.

3.2 El Modelo Consolidado ISO/IEC 14764 e IEEE 1219

Este estándar de la industria conserva los principios de Lientz y Swanson pero introduce una distinción crucial orientada a la ingeniería de calidad y la prevención del riesgo operacional:

- **Preventiva:** Modificación proactiva del código fuente después de la entrega con el propósito de identificar y subsanar defectos latentes o debilidades de diseño estructural antes de que se conviertan en fallas efectivas en el entorno de producción.

Tipo	Criterio	Foco del Cambio	Impacto Funcional
Correctivo	Reactivo ante fallas operacionales	Eliminación de errores lógicos	Restaura el estado óptimo original
Adaptativo	Reactivo ante evolución de TI	Migración de entornos	Usuario percibe idéntica funcionalidad
Perfectivo	Proactivo, alineado al negocio	Nuevas características o mejoras	Expande capacidades del software
Preventivo	Proactivo, control de calidad	Refactorización, deuda técnica	Mejora mantenibilidad futura

Cuadro 1: Clasificación de los tipos de mantenimiento.

4 Comparación de Métricas y Valores Estadísticos

La dimensión económica del mantenimiento revela que la mayor parte del capital invertido en el ciclo de vida del software se consume después de la liberación de la primera versión operativa.

4.1 Evolución Histórica de la Relación de Esfuerzo/Costo

El esfuerzo invertido en mantenimiento ha crecido sustancialmente en las últimas décadas a medida que los sistemas informáticos ganaron escala e interconexión y acumularon capas de código heredado.

Época	Desarrollo	Mantenimiento	Drivers
1970s	60 %-70 %	30 %-40 %	Hardware como mayor inversión
1990s	25 %-35 %	65 %-75 %	ERP multi-módulo, crisis Y2K
2020s	10 %-20 %	80 %-90 %	Legacy crisis, SaaS, deuda técnica

Cuadro 2: Relación esfuerzo desarrollo vs. mantenimiento.

4.2 Distribución Actual del Esfuerzo por Tipo de Mantenimiento

Las estadísticas sectoriales derriban el mito común de que el mantenimiento se reduce simplemente a “reparar errores”, demostrando que la mayor parte de los recursos humanos y financieros se destina a la evolución funcional y la adaptación técnica.

Tipo	% Esfuerzo	Origen del Costo
Perfectivo	25 %-50 %	Nuevos requerimientos de negocio
Adaptativo	15 %-50 %	Actualizaciones de plataformas
Correctivo	20 %-25 %	Diagnóstico de fallas operativas
Preventivo	4 %-15 %	Saneamiento arquitectónico

Cuadro 3: Distribución del esfuerzo por tipo de mantenimiento.

4.3 Costos Financieros y Gestión de Personal en Grandes Empresas

Las decisiones presupuestarias en organizaciones medianas y grandes se rigen por métricas rigurosas de sostenimiento operativo:

- **El Presupuesto “Keeping the Lights On” (KLO):** Las grandes corporaciones gastan de manera recurrente entre el 55 % y el 80 % de su presupuesto global de TI en mantener operativos sus sistemas heredados existentes, dejando un margen sumamente estrecho para la innovación.
- **La Regla del 15–25 %:** Anualmente, el presupuesto requerido para mantener una aplicación en estado óptimo equivale a una tasa fija del 15 % al 25 % de su costo de desarrollo inicial. Para sistemas con altos niveles de deuda técnica, este valor se eleva rápidamente a rangos del 30 % al 40 % anual.
- **Costo Mensual de Equipos de Ingeniería:** El sostenimiento de aplicaciones en producción se factura de acuerdo con la escala del equipo. Un equipo ágil mediano (de 4 a 6 ingenieros full-time) representa un desembolso operativo regular de \$40,000 a \$80,000 mensuales.
- **Costo por Tiempo de Inactividad:** Los fallos en el mantenimiento de sistemas críticos acarrear penalizaciones financieras extremas. Para una gran corporación, la caída del sistema genera pérdidas estimadas entre \$2,300 y \$9,000 por minuto, mientras que para pequeñas empresas el impacto oscila entre \$137 y \$427 por minuto.

5 Acciones, Tareas y Responsabilidades de los Actores de Mantenimiento

Para que el mantenimiento de software sea un proceso repetible, ordenado y predecible, los diversos miembros del equipo de ingeniería deben asumir tareas específicas que resguarden la integridad del sistema.

Actor	Tarea Clave	Tipos	Descripción
Responsable de Mant.	Gestión de Solicitudes (MR/PR)	Todos	Prioriza colas de trabajo y asigna tareas con Scrum o Kanban
Responsable de Mant.	Planificación de Transición	Todos	Evalúa documentación y pruebas para pasar de desarrollo a mantenimiento
Analista de Mant.	Análisis de Impacto y Viabilidad	Correctivo, Adaptativo, Perfectivo	Evalúa componentes afectados para prevenir regresiones
Programador / Técnico	Codificación de Parches	Correctivo, Preventivo, Perfectivo	Hotfixes, actualización de dependencias y optimizaciones
Programador / Técnico	Pruebas de Regresión	Todos	Pruebas automatizadas para validar comportamiento histórico
Programador / Técnico	Migración y Despliegue	Adaptativo	Traslado del sistema a nuevos servidores o nubes

Cuadro 4: Actores del mantenimiento y sus responsabilidades.

6 Glosario Técnico y Análisis de Conceptos Asociados

El estudio científico del mantenimiento descansa sobre una serie de términos técnicos interrelacionados que explican por qué las modificaciones en sistemas complejos pueden volverse sumamente costosas si no se ejerce una disciplina rigurosa de control de calidad interno.

6.1 Mantenibilidad (Maintainability)

De acuerdo con las especificaciones del estándar internacional ISO/IEC 9126 (e integrado de manera proactiva en el proceso de mantenimiento de la ISO/IEC 14764), la

mantenibilidad es la característica cualitativa del software que describe la facilidad, facilidad de análisis y velocidad con la que un sistema de software puede modificarse después de su entrega operacional. Se descompone en cinco subcaracterísticas medibles:

- **Analizabilidad:** capacidad de diagnosticar causas de fallo.
- **Cambiabilidad:** facilidad de implementar modificaciones.
- **Estabilidad:** resistencia a efectos secundarios no deseados.
- **Testabilidad:** verificación eficiente de cambios.
- **Cumplimiento de mantenibilidad:** adhesión a estándares.

6.2 Efecto Dominó (Ripple Effect)

El “efecto dominó” es el fenómeno técnico de propagación de cambios no planificados a través de un sistema de software, en el cual una modificación realizada en un módulo de código específico (módulo A) desencadena de manera inevitable errores o inconsistencias en cascada en múltiples componentes interdependientes (B, C, D) que no estaban originalmente destinados a cambiar.

El término fue acuñado inicialmente por Haney a través de técnicas probabilísticas de matrices de interconexión modular y refinado posteriormente en algoritmos de análisis estático desarrollados por Yau, Collofello y Black. Un sistema propenso al efecto dominó incrementa drásticamente los esfuerzos de mantenimiento y destruye la confiabilidad del producto.

6.3 Estabilidad de un Diseño de Software (Design Stability)

La estabilidad de un diseño es la capacidad de la arquitectura lógica del sistema para resistir la propagación de cambios, es decir, su inmunidad estructural contra el efecto dominó. Un diseño altamente estable confina el impacto de una alteración de manera estrictamente local. Esta propiedad se obtiene en la fase temprana de desarrollo mediante la implementación de principios de bajo acoplamiento, alta cohesión y el empleo estratégico de patrones de diseño orientados a interfaces abstractas que actúan como “escudos” que minimizan las dependencias directas de implementación entre clases de software.

6.4 Sistema Heredado o Código Heredado (Legacy Systems)

Un sistema heredado es una infraestructura informática de software o hardware que, a pesar de operar bajo plataformas obsoletas, tecnologías en desuso y arquitecturas degradadas, resulta indispensable para el funcionamiento diario y la supervivencia del negocio. Estos sistemas suelen presentar una combinación crítica de factores de riesgo: ausencia absoluta de documentación técnica actualizada, pérdida de la traza de los programadores

originales, imposibilidad de realizar pruebas automatizadas y un acoplamiento extremo que los vuelve sumamente vulnerables al efecto dominó.

Su reemplazo total representa un costo financiero y un riesgo de falla de migración prohibitivos, obligando a las organizaciones a extender su ciclo de vida útil a través de costosos parches de mantenimiento.

6.5 Interdependencia Arquitectónica e Impacto de Negocio

La relación que vincula a estos conceptos constituye una cadena de causa y efecto que determina la viabilidad financiera de los proyectos tecnológicos corporativos:

**Diseño Deficiente → Alto Acoplamiento → Efecto Dominó → Baja
Mantenibilidad → Sistema Heredado → Alto Costo**

Un diseño original deficiente, caracterizado por una baja estabilidad de diseño (debido al alto acoplamiento estructural entre clases), propiciará un severo efecto dominó ante cualquier mínima solicitud de cambio correctivo o perfectivo. Esto deteriora significativamente la mantenibilidad global del producto, extendiendo los tiempos requeridos para realizar análisis de impacto (analizabilidad) y exigiendo ciclos redundantes de pruebas manuales de regresión (testabilidad) para evitar caídas en producción.

Con el transcurso de los años, la inserción apresurada de parches puntuales deteriora la arquitectura interna, erosionando la calidad interna del código hasta transformar el software en un sistema heredado o código heredado (“Legacy Code”). En este estado, el costo anualizado de mantenimiento se eleva de manera exponencial, consumiendo la mayor parte de los recursos humanos y financieros de la organización (el presupuesto “Keeping the Lights On”) y bloqueando la capacidad competitiva de la empresa para desplegar innovaciones de negocio.

7 Conclusión / Valoración Personal

Antes de realizar esta investigación, mi idea del mantenimiento de software era bastante limitada: pensaba que básicamente se trataba de corregir errores, eliminar código duplicado e implementar las funcionalidades nuevas que pidiera el cliente. En otras palabras, lo veía como una tarea secundaria dentro del desarrollo.

Sin embargo, al profundizar en el tema, lo que más me sorprendió fue descubrir que la mayor parte del trabajo de un desarrollador no es crear software nuevo, sino mantener el que ya existe. Esto cambió completamente mi perspectiva, porque uno suele imaginarse al programador siempre construyendo cosas nuevas, y la realidad es muy distinta.

Considero que el mantenimiento de software es sumamente importante, principalmente por dos razones. La primera es la escalabilidad del proyecto: un código bien mantenido, que sea fácil de leer y modificar, permite que los desarrolladores pierdan menos tiempo

tratando de entender lo que otros hicieron, lo que reduce errores y frustraciones. La segunda es económica: cuanto más se traba el desarrollo por culpa de un código mal mantenido, más se demoran las entregas, lo que retrasa los pagos y deteriora la imagen del equipo frente a los clientes. Además, si estamos estancados en un proyecto por la mala calidad de su código, menos proyectos nuevos podemos arrancar, y por lo tanto menos ingresos.

A pesar de reconocer su importancia, debo admitir que no me gustaría dedicarme al mantenimiento de software. Prefiero enfocarme en la creación de nuevos sistemas y dejar que otros se encarguen de mantenerlos. Valoro haber aprendido sobre esta disciplina porque me ayuda a entender la industria, pero mi interés está del lado del desarrollo.